

Retrieve Anything To Augment Large Language Models

Peitian Zhang^{✦✦}, Shitao Xiao[✦], Zheng Liu[✦], Zhicheng Dou[✦], Jian-Yun Nie[✦]

✦: BAAI, ✦: Renmin University of China, ✦: University of Montreal ^{*†}

namespace.pt@gmail.com, stxiao@baai.ac.cn, zhengliu1026@gmail.com, dou@ruc.edu.cn, nie@iro.umontreal.ca

ABSTRACT

Large language models (LLMs) face significant challenges stemming from their inherent limitations in knowledge, memory, alignment, and action. These challenges cannot be addressed by LLMs alone, but should rely on assistance from the external world, such as knowledge base, memory store, demonstration examples, and tools. Retrieval augmentation stands as a vital mechanism for bridging the gap between LLMs and the external assistance. However, conventional methods encounter two pressing issues. On the one hand, the general-purpose retrievers are not properly optimized for the retrieval augmentation of LLMs. On the other hand, the task-specific retrievers lack the required versatility, hindering their performance across the diverse retrieval augmentation scenarios.

In this work, we present a novel approach, the **LLM-Embedder**, which comprehensively supports the diverse retrieval augmentation needs of LLMs with one unified embedding model. Training such a unified model is non-trivial, as various retrieval tasks aim to capture distinct semantic relationships, often subject to mutual interference. To address this challenge, we systematically optimize our training methodology. This includes reward formulation based on LLMs' feedback, the stabilization of knowledge distillation, multi-task fine-tuning with explicit instructions, and homogeneous in-batch negative sampling. These optimization strategies contribute to the outstanding empirical performance of the LLM-Embedder. Notably, it yields remarkable enhancements in retrieval augmentation for LLMs, surpassing both general-purpose and task-specific retrievers in various evaluation scenarios. Our checkpoint and source code are publicly available at <https://github.com/FlagOpen/FlagEmbedding>.

KEYWORDS

Large Language Model, Retrieval Augmentation

ACM Reference Format:

Peitian Zhang, Shitao Xiao, Zheng Liu, Zhicheng Dou, Jianyun Nie. 2018. Retrieve Anything To Augment Large Language Models. In *Proceedings of ACM Conference (Conference'17)*. ACM, New York, NY, USA, 16 pages. <https://doi.org/XXXXXXX.XXXXXXX>

^{*}Peitian Zhang and Shitao Xiao contribute equally to this work.

[†]Zheng Liu and Zhicheng Dou are the corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2018 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00

<https://doi.org/XXXXXXX.XXXXXXX>

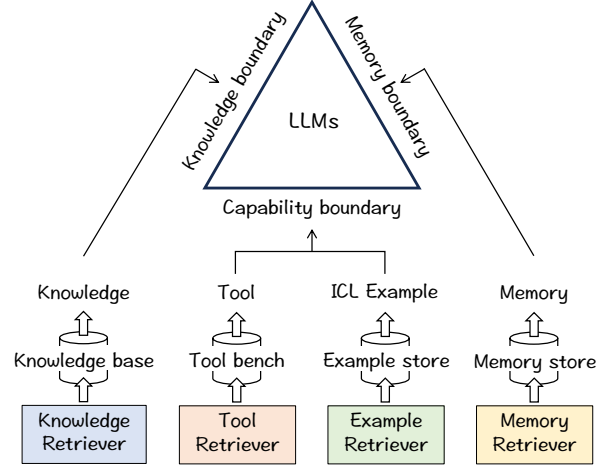


Figure 1: Confront the threefold inherent boundaries of LLMs on top of retrieval augmentation.

1 INTRODUCTION

Large language models represent a significant milestone in the development of general artificial intelligence [17, 19, 78]. While these models have demonstrated unprecedented performance across various general tasks, they still face a series of challenges, including issues such as hallucination [10, 33], instruction following [7, 58], and handling long contexts [2, 8]. Many of these challenges can be traced back to the inherent limitations of LLMs, with three critical boundaries deserving attention.

• **Knowledge boundary.** LLMs are constrained by their knowledge capacity. Due to finite model parameters, they cannot fully internalize the vast body of world knowledge. Moreover, the internal knowledge of LLMs is static and difficult to be updated with the dynamically evolving world. Furthermore, LLMs are predominantly trained on publicly available, high-frequency data, which may result in inaccuracies when dealing with domain-specific or long-tail knowledge.

• **Memory boundary.** LLMs also grapple with severe limitations in memory, primarily due to restrictions on context length. While advances have been continually made in expanding the maximum context length, it still falls short of achieving the goal of lifelong engagement with human users. Additionally, both the training and deployment of LLMs with extended context can be prohibitively computationally and storage-intensive, making it impractical to significantly expand their memory.

• **Capability boundary.** LLMs' capabilities are constrained in terms of action and autonomy. Firstly, they are limited to the 'language space' and cannot meaningfully interact with the physical world. Secondly, these models heavily rely on human guidance, requiring clear user instructions and appropriate demonstration examples to perform specific tasks effectively.

The above inherent boundaries cannot be effectively addressed by LLMs alone. To overcome these limitations, external assistance is sought through the process known as retrieval-augmented generation [15, 27, 32, 41]. Retrievers play a crucial role in connecting LLMs with the necessary external components, enabling LLMs to accomplish various downstream tasks (see Figure 1). In this context, several common types of retrievers have been designed, each tailored to fulfill a distinct role in enhancing LLMs:

- **Knowledge Retriever:** providing external knowledge to support LLMs in tackling knowledge-intensive tasks [37, 41, 59].
- **Memory Retriever:** collecting information that extends beyond the immediate context, assisting in the generation of lengthy sequences [12, 71, 85].
- **Tool Retriever:** selecting appropriate tools, allowing LLMs to interact effectively with the physical world [61, 62, 74].
- **Example Retriever:** locating pre-cached demonstration examples, from which LLM prompts can be automatically generated to facilitate in-context learning [47, 83].

Given the importance to connect LLMs with the external world, it is imperative to optimize the performance across various tasks. The effectiveness of retrieval systems heavily rely on the quality of embeddings [30, 37, 68, 92]. Consequently, the optimization challenge centers around the learning of embedding model. Historically, two common approaches have been employed. The first approach focuses on developing task-specific models, where the embeddings are tailored for specific applications, such as question answering [96] or in-context learning [83]. While this approach leads to a competitive performance within each scenario, it lacks the versatility across different contexts. In contrast, the second approach resorts to general-purpose embedding models [59, 60], which aim to be universally applicable [30, 82, 89]. However, these methods are not properly optimized for the specific requirements of retrieval augmentation for LLMs. This limitation significantly hampers their performance in corresponding tasks.

In this work, we propose LLM-Embedder, a unified embedding model to satisfy primary retrieval augmentation needs of LLMs. Unifying the diverse retrieval capabilities holds significant advantages. From a practical standpoint, LLM-based systems often require multiple external modules, such as knowledge bases, memory stores, and tool-bench, to execute complex tasks. By consolidating these functionalities into a unified model, we can streamline system management and enhance operational efficiency. From the perspective of effect, the unified model may also benefit from the composite data of different scenarios. This can be especially helpful for the retrieval tasks where high-quality training data is scarce.

However, training a unified model poses substantial challenges. Firstly, the embedding model must optimize its ultimate impact on retrieval augmentation, instead of focusing solely on intermediate retrieval results. Secondly, the diverse retrieval tasks seek to capture distinct semantic relationships, which may not always be mutually beneficial but sometimes interfere with each other. To address both challenges, we optimize our training methodology as follows.

- **Reward from LLM.** To train the LLM-Embedder, we utilize a combination of labels from various sources. In addition to the native hard labels from the original datasets, we leverage rewards obtained from the LLM's output. A retrieval candidate is assigned

a higher reward if it substantially improves the LLM's final performance. These rewards are considered soft labels and are learned via knowledge distillation by the embedding model.

- **Stabilized distillation.** Given the diversity of training data, the LLM's output can exhibit significant fluctuations. In some cases, the output scores may be distributed too closely or polarized, making it challenging to assess the fine-grained quality of candidates. To mitigate this issue, we introduce stabilized distillation. It jointly incorporates soft reward-based labels and hard ranking-based labels, where the distillation effect is significantly improved.

- **Instruction based fine-tuning.** We curate a diverse training dataset comprising a wide variety of tasks closely related to the retrieval augmentation for LLMs. To harmonize the training impact across different data sources, we take advantage of instruction based fine-tuning, where task-specific prompts are used to differentiate each individual task [5, 76].

- **Homogeneous in-batch negative sampling.** In-batch negative sampling is a common practice to introduce a large number of negative samples [37, 63]. However, one potential drawback in our context is that negative samples shared across different tasks (i.e. heterogeneous negatives) may be less effective in discriminating semantic relationships for a specific context. To mitigate this issue, we construct each mini-batch using training data from the same tasks, ensuring that the in-batch negatives are homogeneous and contribute effectively to the discriminative power of embeddings.

To summarize, our work makes significant contributions in the following ways.

- **LLM-Embedder:** We introduce LLM-Embedder, a novel embedding model designed to bridge LLMs with the external world. To the best of our knowledge, LLM-Embedder is the first of its kind, offering comprehensive support for all key facets of LLMs' retrieval augmentation.
- **Systematic Optimization:** We systematically optimize LLM-Embedder across multiple dimensions, including reward formulation, knowledge distillation, instruction based fine-tuning, and negative sampling, which ensures the effectiveness of the proposed model.
- **Empirical Validation:** We verify the effectiveness of LLM-Embedder with comprehensive experiments. Our model outperforms the existing embedding models, significantly amplifying the impact of retrieval augmentation on various critical aspects of LLMs, such as knowledge enhancement, long-context modeling, and instruction following.

2 LLM-EMBEDDER

The introduction of LLM-Embedder is partitioned into the following three parts: 1) the curation of training data, 2) the training methodology, 3) the retrieval augmentation of LLMs.

2.1 Training Data

LLM-Embedder is to serve as a unified model for the retrieval augmentation of LLMs. To fulfill this objective, we assemble a diverse training dataset from the following tasks. 1) Question Answering. We utilize MSMARCO [57] and Natural Questions [39] to establish the model's knowledge retrieval capability. 2) Conversational Search. The QReCC dataset [3] is employed to further improve

the model’s information seeking capability in the conversational context. 3) Tool Learning. The ToolLLM dataset [62] is used to learn the selection of appropriate tools in the tool-using context. 4) Instruction Tuning: To retrieve useful demonstration examples for in-context learning, we re-purpose FLAN [86] and UPRISE [18], which are originally designed for instruction tuning. 5) Generation. The model is trained to extract valuable historical information (i.e. memory) based on a long conversation dataset: Multi-Session Chat [93], as well as long-range language modeling datasets: including Books3 [25], ArXiv [25], CodeParrot [79]. These datasets can be grouped into two types based on the availability of labels.

- **Labeled data.** The datasets on the first three types of tasks are composed of pairwise texts, where hard-coded labels are presented. For question answering datasets (MSMARCO, NQ), each data instance consists of a query and the source passage of answer, denoted as $\langle \text{query}, \text{passage} \rangle$. For conversational search dataset (QReCC), each data instance is made up of a conversational query and the source passage of answer, denoted as $\langle \text{conversation}, \text{passage} \rangle$. For tool learning dataset (ToolLLM), each data instance includes an instruction and the description of the needed tool, denoted as $\langle \text{instruction}, \text{tool desc} \rangle$.

- **Non-labeled data.** In contrast, the last two types of datasets do not have explicit labels. For instruction tuning datasets (FLAN, UPRISE), each instance consists of human’s instruction and the expected output: $\langle \text{instruction}, \text{output} \rangle$. For generation datasets, each instance is a long text sequence partitioned into chunks: $[\text{chunk}_0, \dots, \text{chunk}_L]$. Books3, ArXiv, and CodeParrot are made up of plain texts, which are chunked into spans of equal length (128 tokens per chunk). Multi-Session Chat is composed of conversations, where each chunk corresponds to a pair of consecutive utterances.

2.2 Training Methodology

2.2.1 Formulation of Training Reward. In our work, we explore two types of supervision signals for training the LLM-Embedder. Firstly, we can directly utilize the hard labels provided by the labeled datasets. Secondly, we aim to optimize the LLM’s final performance with retrieval augmentation. To achieve this goal, we leverage the **reward produced by LLM** for both labeled and unlabeled datasets. Particularly, given the expected output of the LLM, denoted as O , and a retrieval candidate, denoted as C , the reward for the candidate, represented as $r_{C|O}$, is derived by the following equation:

$$r_{C|O} = \prod_{i=1}^{|O|} \text{LLM}(o_i|C, O_{i-1}). \quad (1)$$

Here, o_i represents the i -th token of the expected output, and $\text{LLM}(x|y)$ stands for the LLM’s generation likelihood of producing x given the context y . In other words, a higher reward is assigned to a retrieval candidate if it results in a higher generation likelihood for the expected output.

The LLM based reward is applied in the following ways for each of the tasks in consideration. 1) For Question Answering: the reward is computed as the generation likelihood of answers given one single candidate passage. 2) For Instruction Tuning: The reward is computed as the generation likelihood of the instructed output given one candidate example. 3) For Generation: the reward is computed as the generation likelihood of a new content given one candidate historical chunk. Note that the LLM reward is not

applied to conversational search and tool learning datasets, as there is no clear expectation of the LLM’s output in these cases.

Given the two sources of supervision signals of LLM-Embedder, i.e. the native hard labels and the soft reward derived from LLM, the training is conducted with a composite recipe. The contrastive learning is applied to capture the semantic relationship reflected by the hard labels; meanwhile, the knowledge distillation is used to learn from the soft rewards derived from LLM.

2.2.2 Contrastive Learning. For each pair of hard-labeled texts: q and p (e.g., query and passage), the loss function of contrastive learning is formulated in the following way:

$$\min_{(q,p)} \sum_{(q,p)} -\log \frac{\exp(\langle \mathbf{e}_q, \mathbf{e}_p \rangle / \tau)}{\sum_{p' \in \mathcal{P}} \exp(\langle \mathbf{e}_q, \mathbf{e}_{p'} \rangle / \tau)}, \quad (2)$$

where $\mathbf{e}_*$ stands for the embedding, $\langle \cdot \rangle$ indicates the inner product operator, $\mathcal{P}$ are the union of positive and negative samples, τ refers to the temperature. To improve the discriminative power of embeddings across diverse application scenarios, we employ a couple of key designs in our contrastive learning framework.

The first featured design is the **Instruction-based Fine-Tuning**. In this approach, each task is assigned with a unique task instruction denoted as I_t . While generating the query-side embedding, the task instruction and query content are concatenated and jointly encoded, resulting in the update of query embedding: $\mathbf{e}_q \leftarrow \text{encode}([I_t, q])$. This task-specific instructions plays a pivotal role in initializing the embedding model with distinct activations, thereby facilitating the discrimination between different tasks.

The second notable feature is the **Homogeneous In-Batch Negative Sampling**. It calls for a considerable amount of negative samples to guarantee the embedding’s discriminativeness [30, 63, 82]. In our work, this is realized by the joint usage of in-batch negatives and hard negatives. We also apply cross-device sharing [63, 91], which further expands the scale of negative samples. Consequently, our method results in $B \times K \times N - 1$ negative samples in total, where B is the batch size, K is the number of GPU devices, N is the total number of positive and hard negative samples. However, the vanilla practice of in-batch negative sampling presents one drawback in our multi-task settings. Particularly, the embeddings shared between different datasets (namely heterogenous negative samples) are mostly irrelevant, which are less effective for discriminating the semantic relationships within a specific task scenario. To address this limitation, we introduce a regularization strategy for the organization of training data, where the data instances from the same task are grouped into consecutive mini-batches. The strategy makes the majority of in-batch negative samples to originate from the same dataset (i.e. homogeneous negative samples), thus enhancing the discriminative power of embeddings for each specific task.

2.2.3 Knowledge Distillation. In our training framework, knowledge distillation plays a crucial role in learning from the LLM’s reward. we employ the KL-divergence to minimize the gap between the distributions of candidates computed using LLM’s rewards and those predicted by the embedding model. In particular, for each query q and its candidate list $\mathcal{P}$: $[p_1, \dots, p_N]$, we derive the LLM’s rewards towards the candidates, denoted as R : $[r_1, \dots, r_N]$, using

Eq 1. To make the LLM’s rewards suitable for distillation, we transform each reward into a normalized weight: $w_i \leftarrow \text{softmax}_R(r_i/\alpha)$, where α represents the temperature. On top of these elements, the KL divergence is computed by the following equation:

$$\min. \sum_{\mathcal{P}} -w_i * \log \frac{\exp(\langle \mathbf{e}_q, \mathbf{e}_p \rangle / \tau)}{\sum_{p' \in \mathcal{P}} \exp(\langle \mathbf{e}_q, \mathbf{e}_{p'} \rangle / \tau)}. \quad (3)$$

While the above formulation has been successfully employed in mono-task settings [29, 46, 81], applying it directly to our multi-task scenario poses unique challenges. Notably, the magnitude of LLM’s rewards can exhibit high fluctuations due to the diverse training samples from various tasks. In many cases, the LLM’s rewards closely distribute, making it challenging to distinguish the quality of candidates. In contrast, in many other cases, the rewards become polarized, with candidates receiving either a positive reward or nearly zero rewards. Both of these scenarios contribute little to the distillation process and can severely impair the training effect.

• **Stabilized Distillation.** To address the challenge of fluctuated rewards in our multi-task scenario, we introduce a modified formulation of the loss function. This adaptation effectively alleviates the negative impact resulted from the rewards’ fluctuations. Particularly, instead of using LLM rewards solely as “soft weights”, we also leverage them as hard ranking labels. Given LLMs’ rewards $R: [r_1, \dots, r_N]$, we re-rank the candidates in a top-down order. This operation results in a new order for the candidates, denoted as $\mathbb{P}: [p_1, \dots, p_N]$, where $r_i \geq r_{i+1}$. The loss function for knowledge distillation is accordingly transformed as follows:

$$\min. \sum_{\mathcal{P}} -w_i * \log \frac{\exp(\langle \mathbf{e}_q, \mathbf{e}_{p_i} \rangle / \tau)}{\sum_{p' \in \mathbb{P}} \exp(\langle \mathbf{e}_q, \mathbf{e}_{p'} \rangle / \tau)}.$$

Here, $\mathbb{P}$ comprises two sources: the lower-ranked candidates of $p_i: [p_{i+1}, \dots, p_N]$; and the the in-batch negative samples.

Our adapted formulation serves to stabilize fluctuated rewards in two fundamental ways. On one hand, the model is consistently trained to promote p_i compared to its lower-ranked counterparts $[p_{i+1}, \dots, p_N]$. This means that the model is always able to learn from the LLMs’ preferences, regardless of the absolute value of rewards. This mechanism is particularly effective in handling cases where LLMs’ rewards are too closely distributed. On the other hand, when the top-ranked candidate receives a significantly higher reward compared to the other candidates, the weights will become one-hot. In this scenario, the distillation process will be reduced to the form of contrastive learning, with the top-ranked candidate treated as a positive sample. This mechanism help to address the situations where polarized rewards are generated by LLMs.

2.3 Retrieval Augmentation of LLMs

The multi-tasking capacity of LLM-Embedder makes it as a versatile solution. By connecting to the vector DB where any needed external elements are stored, it may support a wide variety of retrieval augmentation tasks. In this place, we discuss the typical scenarios empowered by LLM-Embedder (Figure 2), with focusing on three key issues: 1) what to store in the vector DB, 2) what is used to query the vector DB, 3) how to leverage the retrieved data.

• **Knowledge Enhancement.** When handling knowledge intensive tasks [37, 59], the entire docs from the knowledge corpus can be encoded and stored in vector DB. In many cases, questions

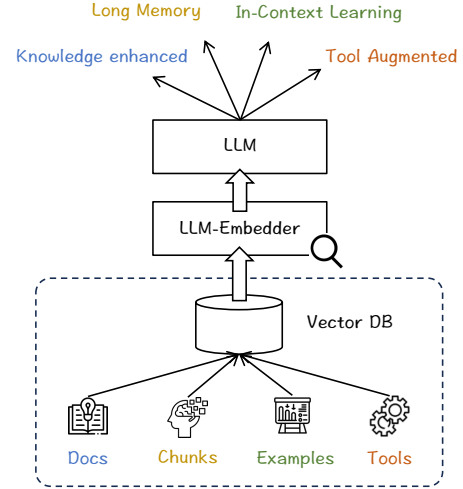


Figure 2: Retrieval augmentation with LLM-Embedder.

are explicitly presented, which can be used to query the vector DB. In other cases, the working context during the generation process can be used as query [27, 34]. The retrieved docs can be directly applied or refined for more informative segments [44]. Finally, the query and retrieved docs are concatenated to generate knowledge-grounded answer, e.g., [knowledge, query] $\rightarrow$ answer.

• **Long-Context Modeling.** When dealing with a long context, the entire history can be chunked, encoded, and off-loaded to the vector database. The working context during the generation process can be used to query the vector DB for relevant chunks. In many cases, both the relevant chunk, e.g., chunk_{*i*}, and its subsequent chunk_{*i+1*} are used for memory augmentation [15], because the subsequent chunk can be more critical to the future generation. The retrieved chunks are used to back-fill the current context, where new content can be generated with remote but important memory, e.g., [retrieved chunks, current context] $\rightarrow$ new generation.

• **In-context Learning.** The demonstration examples, organized in the form of “(task instruction, expected output)”, can be encoded and pre-stocked in vector DB. When a new task is given, the task’s instruction is used to query the vector DB [18, 83]. The retrieved examples are concatenated with the task’s instruction, based on which the in-context learning can be conducted, e.g., [retrieved examples, instruction] $\rightarrow$ task completion.

• **Tool Learning.** The tool’s functionality can be verbalized as a description, and paired with its API: “(description, API)”. In this way, a massive toolkit can be managed by vector DB based on the encoded description [62]. Given a user request that involves the use of tools, the user request can be encoded and used to query the vector DB. The retrieved tool is executed via its API, where the execution result is returned for LLM to complete the remaining generation: [user request, tool’s execution result] $\rightarrow$ generation.

3 EXPERIMENT

The experimental study is to clarify three basic research questions. **RQ 1.** can LLM-Embedder comprehensively support the diverse scenarios of LLM’s retrieval augmentation. **RQ 2.** what is LLM-Embedder’s impact to each specific scenario. **RQ 3.** what are the key factors influencing the empirical performance of LLM-Embedder.

3.1 Settings

The baseline, datasets, evaluation method, and implementation of the experiment are introduced as follows. Given the limited space, more detailed specifications are presented in the Appendix.

3.1.1 Baselines. Firstly, we measure the performance of Language Model Models (LLMs) without retrieval augmentation, denoted as **None**, to gauge the empirical benefits introduced by retrieval augmentation. Secondly, we make comparison with a series of baseline retrievers, which are categorized into two types. 1) **General embedding models.** These models are trained to support a wide range of text retrieval and representation tasks, such as question answering, entity retrieval, duplication detection, and document ranking. Our experiment includes the following widely-recognized baselines: **Contriever** [30], **Instructor** [76], **RetroMAE-BEIR** [46], and **BGE** [89]. These methods are empirically competitive according to BEIR [77] and MTEB [53] benchmarks, among which BGE maintains the leading performance upon the time of this work. 2) **Task-specific embedding models.** These models are tailored to optimize performance on specific tasks. We include the following task-specific baselines, which excel in their respective domains: **ARR** [96] for knowledge enhancement of LLMs, **LLM-R** [83] for in-context learning, **API-Retriever** [62] for tool learning, and **ConvANCE** [49] for conversational search. Additionally, we consider **BM25** [69], a widely used retriever based on lexical similarity.

3.1.2 Evaluation and Datasets. We present the tasks used to assess the retriever’s performance, including knowledge enhancement, in-context learning, long-context modeling, tool learning, conversational information seeking. For each task, we introduce the relevant evaluation dataset and methodology.

- **Knowledge Enhancement.** We adopt the established setup used by AAR [96]. The experiment is performed on two popular benchmarks. 1) **MMLU** [28], which comprises multiple-choice questions evaluated by accuracy. 2) **PopQA** [48]: which involves question answering tasks evaluated by exact match (EM). Following AAR, the knowledge is retrieved from MS MARCO [57] and Wikipedia Corpus [59], respectively.

- **In-Context Learning.** We adopt the data and framework from LLM-R [83]. There are 30 public datasets from 9 distinct categories, including **Close QA (CQA)**, **Commonsense (Comm)**, **Coreference (Coref)**, **Paraphrase (Para)**, **NLI**, **Reading Comprehension (RC)**, **Sentiment Analysis (Sent)**, **Data2Text (D2T)**, **Summarization (Summ)**. To better assess the generalization ability, we withhold four datasets (QNLI, PIQA, WSC273, Yelp) from the training stage. We collect demonstration examples from the combination of FLAN [86] and UPRISE [18], creating a retrieval pool of 6.3 million examples. For each presented task, we retrieve the top-8 examples to complete the task. Each task employs a specific evaluation metric, whose specifications are presented in Appendix.

- **Long-Context Modeling.** We focus on two scenarios: long conversation and long-range language modeling. The first scenario leverages **Multi-Session Chat** [93]. We retrieve historical dialogue turns with the current utterance, append them ahead of the current utterance, based on which the next response is generated. Following existing literature about augmenting memory for LLMs [71, 85, 88], we leverage **Books3** [25], **ArXiv** [25], **CodeParrot** [79], and

PG19 [64] for the second scenario. We hold out PG19 entirely from training to assess the generalization ability. These datasets divide each historical sequence into chunks of 128 tokens. Historical chunks are retrieved based on the latest chunk, appended ahead of the current context, based on which the future chunk is generated. Performance in both scenarios is measured by Perplexity (PPL).

- **Tool Learning.** We follow the established data and framework from ToolLLM [62], whose primary objective is to find the needed tool based on the instructions and the tool’s descriptions. The dataset already provides ground-truth information about the needed tool, which allows us to directly measure the retriever’s performance using its ranking performance, specifically NDCG@5.

- **Conversational Search.** We use the setup of QReCC [3] for evaluation, where the required knowledge is retrieved based on the concatenation of conversation’s context and the last query. Like ToolLLM, this dataset also provides ground-truth, whereby letting the retriever’s performance to be directly measured by its ranking performance (NDCG@3 following previous works [50]).

3.1.3 Implementation. There are two critical factors about the implementation: the LLM foundation and the embedding model backbone. As for LLM Foundation, we choose to work with **Llama-2-7B-Chat** [78] for two reasons: 1) it is a white-box LLM, allowing for easy extraction of reward and perplexity metrics; 2) it’s empirically competitive and relatively lightweight, making it well-suited for our research purposes¹. Given that the maximum sequence length of Llama-2 is 4096 tokens, we retain the latest 2048 tokens and retrieve an additional 2048 tokens from history to assess language modeling performance. As for embedding backbone, we utilize **BGE** base [89] to initialize our model. BGE is well pre-trained with general text embedding tasks, which provides a strong foundation to develop the needed capabilities of LLM-Embedder.

3.2 Analysis

The experiment results are analyzed from three perspectives: the overall analysis, the analysis for each individual scenario, and the ablation studies for influential factors.

3.2.1 Overall Analysis. The experiment results on different retrieval augmentation scenarios are presented with Table 1-3, respectively. We can come to the following conclusions given the observations across all the presented results.

Firstly, compared with the result from plain LLM, i.e. None, LLM-Embedder helps to deliver more precise answers with the retrieved knowledge (Table 1), better instruction following effect with the retrieved examples (Table 2), and improved quality of long-sequence generation with the retrieved memory (Table 3). Besides, the LLM’s performance can also be improved by other baseline retrievers in many of the situations. However, the relative improvements are not always as significant as the ones with LLM-Embedder. Such observations indicate that *LLMs can benefit from properly retrieved assistance; and with a stronger retriever, the augmentation’s impact can be substantially magnified.*

Secondly, LLM-Embedder brings forth a competitive retrieval augmentation effect across the diverse scenarios. On one hand, it

¹Although using rewards from Llama-2 7B Chat, LLM-Embedder is also applicable to other LLMs. Evaluations about this are presented in Appendix.

Table 1: Impact on knowledge enhancement. MMLU and PopQA are measured by precision and exact match (EM), respectively. “*” and “†” indicates the SOTA general embedding model and the task-specific method for the corresponding scenario.

Method	MMLU					PopQA
	STEM	Social	Human	Other	All Avg.	PopQA
None	0.3468	0.5328	0.5094	0.4967	0.4599	0.2061
BM25	0.3760	0.5378	0.5051	0.5088	0.4721	0.3491
Instructor [76]	0.3702	0.5406	0.5111	0.5082	0.4721	0.3533
Contriever [30]	0.3677	0.5383	0.5080	0.5013	0.4684	0.3276
RetroMAE-BEIR [46]	0.3857	0.5456	0.5221	0.5276	0.4853	0.4364
BGE* [89]	<u>0.3852</u>	<u>0.5564</u>	0.5194	0.5389	<u>0.4896</u>	0.4491
AAR† [96]	0.3802	0.5501	0.5125	0.5288	0.4826	<u>0.4792</u>
API-Retriever [62]	0.3535	0.5335	0.4999	0.5068	0.4625	0.2488
LLM-R [83]	0.3629	0.5277	0.5018	0.4984	0.4625	0.2506
LLM-Embedder	0.3848	0.5568	0.5255	<u>0.5360</u>	0.4903	0.5052

notably outperforms a series of general retrievers, including the state-of-the-art method BGE. On the other hand, it also goes beyond the task-specific method, i.e. AAR for knowledge enhancement, LLM-R for in-context learning, API-Retriever for tool learning, Conv-ANCE for conversational search. Such an observation indicates that *LLM-Embedder is able to provide a strong and unified foundation to support different retrieval augmentation needs of LLMs.*

Finally, we can also observe that the task-specific retrievers optimized for one scenario could result in limited performances in other scenarios, indicating that the training impacts between different retrieval tasks are not always transferable. To better illustrate this point, we visualize the retrieval augmentation’s impact (improvements over None) from five representative methods in Figure 3: BGE, AAR, LLM-R, API-Retriever (API-R), and LLM-Embedder (ours). The first method is the general embedding model, while the second to fourth are task-specific methods. We can observe that although task-specific training can deliver a competitive performance for its corresponding scenario, e.g., AAR for knowledge enhancement and LLM-R for in-context learning, their impacts are severely weakened when applied for other usages. In contrast, LLM-Embedder demonstrates a steady and competitive performance across different scenarios. *Although challenging, the seemingly irrelevant or even adverse retrieval patterns can still be reconciled by one unified embedding model on top of the properly optimized training recipe.*

3.2.2 Individualized Analysis. Further analysis is made for the retrieval augmentation’s impact to each individual scenario.

• **Knowledge Enhancement.** The experiment results on knowledge enhancement are shown in Table 1, where we can make the following observations. 1) Benefit of external knowledge. LLMs benefit from external knowledge when answering questions in MMLU and PopQA, as clear empirical advantages are achieved by the retrieval augmentation methods compared with the plain LLM, i.e. None. 2) Importance of retrieval accuracy. The impact of knowledge enhancement becomes more pronounced when knowledge retrieval is more accurate. We observe consistent improvements as we transition from using the BM25 retriever to more advanced embedding models. 3) Distinction between datasets. The impact of retrieval augmentation is more noticeable in the PopQA dataset compared to MMLU. This difference is likely due to the nature of

	QA	ICL	Long	Tool	C-Search
BGE	0.4491	0.5974	14.2943	0.5761	0.3856
AAR	0.4792	0.5938	14.6999	0.4200	0.2877
LLM-R	0.2506	0.6262	14.4746	0.1321	0.0234
API-R	0.2488	0.5945	14.7834	0.8017	0.1137
Ours	0.5052	0.6268	13.4832	0.8645	0.5053

Figure 3: Retrieval augmentation’s impact from different retrievers. The warmer color indicates a better performance.

the datasets. PopQA tends to be more knowledge-intensive, with a focus on questions about long-tail entities. In contrast, many questions in MMLU rely more on common sense and logical reasoning rather than extensive world knowledge.

• **In-Context Learning.** The experiment results on in-context learning are shown in Table 2, where we can draw the following observations. 1) Benefits of retrieved examples. When comparing plain LLM (None) with other retrieval-augmented methods, we can consistently observe the improved performances in most cases. This finding underscores the enhancement of LLM’s ability to follow instructions when retrieved examples are presented. 2) Limitation of BM25. It’s noteworthy that BM25’s performance is comparatively weaker than its performance in other scenarios. This discrepancy can be attributed to the specific nature of in-context learning, where examples need to emphasize semantic similarity rather than lexical similarity. 3) Limited transferability. While the task-specific method, LLM-R, exhibits a competitive performance for in-context learning, its utility becomes severely limited when applied to other scenarios, such as knowledge retrieval and tool using. This suggests that example retrieval calls for a unique pattern tailored to this very task, making it challenging to transfer to other scenarios.

• **Long-Context Modeling.** The experiment results on long-context modeling are shown in Table 3. While retrieval augmentation consistently demonstrates improvements compared to having no augmentation (None), it may not be entirely convincing due to the utilization of more context. To address this issue, we introduce

Table 2: Impact on in-context learning. The performances are measured by Misc. metrics (see Appendix).

Method	In-Context Learning									Avg
	CQA	Comm	Coref	Para	NLI	RC	Sent	D2T	Summ	
None	0.2923	0.7212	0.6578	0.5242	0.4478	0.4892	0.7077	0.1982	0.1447	0.4645
BM25	0.3603	0.7019	0.6029	0.5059	0.4583	0.5396	0.7284	0.3019	0.1555	0.4840
Instructor	0.5003	0.7772	0.5735	0.6312	0.5360	0.6219	0.9148	0.4595	0.4572	0.6036
Contriever	0.4912	0.7723	0.5624	0.6358	0.5466	<u>0.6297</u>	0.9141	0.4380	0.4444	0.6009
RetroMAE-BEIR	0.4594	0.7742	0.5840	0.5755	0.5408	0.6029	0.9286	0.4661	0.4465	0.5939
BGE*	0.4718	0.7773	0.5550	0.6171	0.5413	0.5988	<u>0.9281</u>	0.4719	0.4521	0.5974
AAR	0.4809	0.7796	0.5848	0.5890	0.5354	0.6039	0.9210	0.4445	0.4410	0.5938
API-Retriever	0.4765	0.7620	0.5465	0.6266	0.5204	0.6096	0.9245	0.4866	0.4424	0.5945
LLM-R [†]	0.5165	<u>0.7802</u>	0.5830	0.6567	0.6145	0.6223	0.9059	<u>0.4777</u>	0.4878	<u>0.6262</u>
LLM-Embedder	<u>0.5163</u>	0.7842	<u>0.5927</u>	<u>0.6556</u>	<u>0.6041</u>	0.6318	0.9224	0.4731	<u>0.4742</u>	0.6268

Table 3: Impact on long conversation and language modeling (PPL), tool learning (NDCG), conv search (NDCG).

Method	Conversation	Language Modeling				Tool	C-Search
	MSC	Books3	Arxiv	CodeParrot	PG19 (o.d.)	ToolLLM	QReCC
None	19.3501	8.8193	3.7647	2.7663	10.2510	–	–
Recency	13.9569	8.7391	3.4158	2.5989	10.2216	–	–
BM25	14.6512	8.6576	3.3106	2.4591	10.1960	0.5115	0.4341
Instructor	14.8799	8.6619	3.3546	2.4756	10.2011	0.3882	0.2863
Contriever	14.2129	8.6460	3.2709	2.4437	10.1616	0.4904	0.3563
RetroMAE-BEIR	14.3990	8.6376	3.2903	2.4592	10.1731	0.5205	0.4037
BGE*	14.2943	8.6311	3.2912	2.4578	10.1541	0.5761	0.3856
AAR	14.6999	8.6381	3.3260	2.4666	10.1808	0.4200	0.2877
API-Retriever [†]	14.7834	8.6722	3.3858	2.4919	10.1833	<u>0.8017</u>	0.1137
Conv-ANCE [†]	–	–	–	–	–	–	<u>0.4560</u>
LLM-R	14.4746	8.6619	3.3635	2.4724	10.2024	0.1321	0.0234
LLM-Embedder	13.4832	8.6080	3.2322	2.4303	10.1185	0.8645	0.5053

a simple yet strong baseline called Recency. Rather than using retrieved context, Recency directly leverages the most recent context immediately preceding the current window. For example, in conversation, it considers the last pair of utterances before the current session; and in language modeling, it introduces the content within the range of 2049-4096 tokens preceding the latest 2048 tokens.

With the introduction of this new baseline, the impact of retrieval augmentation becomes more nuanced. On one hand, the LLM-Embedder continues to exhibit superior performance across various situations. On the other hand, other retrievers no longer guarantee a consistent enhancement: although alternative retrieval-augmented methods yield improved generation quality for language modeling, a majority of them fall short of Recency’s performance while dealing with conversation. This observation underscores the challenges regarding effective memory retrieval in practice.

• **Tool Learning and Conversation Search.** The experiment results on tool learning and conversational search are shown in Table 3. In line with our prior observations, the task-specific approaches, i.e. the API retriever (Tool) and Conv-ANCE (Conv Search), consistently deliver higher performances than most of the baselines. Besides, unlike other cases, BM25 overtakes most of the embedding models in these two scenarios. However, it’s worth noting that

LLM-Embedder continues to maintain the leading position, which again highlights its capability in unifying diverse retrieval tasks.

3.2.3 Ablation Studies. The ablation studies are presented to analyze the influential factors about LLM-Embedder’s training process (see Table 4): reward from LLM, instruction based fine-tuning, homogeneous in-batch negative sampling, and stabilized distillation.

For “**w.o. LLM reward**”, we replace the soft reward from LLM by using highest rated candidates as positive samples (i.e. hard labels). By doing so, the knowledge distillation is reduced to contrast learning. The empirical performance in most of the scenarios are decreased due to such a change. However, the performances in tool learning and conversational search are little affect; this is comprehensible knowing that LLM-Embedder is purely trained with hard labels in both scenarios.

For “**w.o. instruction FT**”, we remove the task-specific instructions while fine-tuning LLM-Embedder. Without such a component, it will become harder for the embedding model to discriminate the retrieval task in different scenarios. This speculation is consistent with the observed result, as LLM-Embedder’s performance is decreased from such a change.

For “**w.o. homo NS**”, the homogeneous in-batch negative sampling is disabled. Such a change could reduce the discrimination of

Table 4: Ablation study for the three influential factors about LLM-Embedder’s training: using soft reward from LLM, stabilized distillation, instruction based fine-tuning, in-batch negative sampling from the same scenario.

Method	Knowledge		ICL	Long		Tool	Conv Search
	MMLU	PopQA	Misc.	MSC	ArXiv	ToolLLM	QReCC
w.o. LLM Reward	0.4872	0.4794	0.6217	13.9176	3.2495	0.8927	0.4945
w.o. Instruction FT	0.4776	0.5025	0.6211	13.9125	3.2383	0.8192	0.5029
w.o. homo NS	0.4791	0.4520	0.6200	14.0441	3.2558	0.8364	0.4563
w.o. Stabilized Distill	0.4815	0.5027	0.6105	13.6090	3.2441	0.7905	0.4865
AAR	0.4826	0.4792	0.5938	14.6999	3.3260	0.4200	0.2877
API-Retriever	0.4625	0.2488	0.5942	14.7834	3.3858	0.8017	0.1137
LLM-R	0.4625	0.2506	0.6262	14.4746	3.3635	0.1321	0.0234
LLM-Embedder	0.4903	0.5052	0.6268	13.4832	3.2322	0.8645	0.5053

the embeddings, because a great portion of the negative samples will come from different tasks, which are irrelevant with each other. As we can observe, LLM-Embedder’s performance is decreased due to such a change, especially for PopQA and Conv Search, where a massive candidate pool is presented (Wikipedia corpus).

For “w.o. stabilized distill”, we replace our stabilized distillation with the conventional KL-divergence based method. As introduced, this operation handles the fluctuated reward from LLM such that distillation can become more stabilized. We can observe that LLM-Embedder’s performance is reduced once this step is removed, especially for ICL where LLM’s reward is the major training signal.

4 RELATED WORKS

The related works are reviewed from two perspectives: retrieval augmented large language models, and dense retrieval.

• **Retrieval Augmented LLMs.** Large language models (LLMs) are praised for their unprecedented capability on language understanding and generation. Compared with the conventional methods, LLMs exhibit overwhelming generality and notable advantages on typical NLP tasks [17, 19, 78]. Despite such superiority, LLMs still face a series of severe challenges, such as hallucination, human alignment, and long-term memory. Many of the existing problems are caused by the inherent boundaries, which cannot be addressed by LLMs alone, but to rely on support from the external world. The retrieval-augmented LLMs are regarded as a go-to option to bridge LLMs with the external assistance [4, 51]. For the past few years, they have been widely applied to several critical scenarios. One common case is the knowledge enhancement. The internal knowledge of LLMs can be incomplete, static, and limited by the popularity bias. When dealing with knowledge intensive tasks, the retrieval augmented LLMs will look for necessary information from an external database, where the generated content can be grounded on proper knowledge [15, 31, 32, 41]. Besides, the retrieval augmented LLMs are also used to retrieve historical context to establish long-term memory [71, 85], retrieve examples to improve the instruction following capability [18, 83], and retrieve tools to engage with the physical world [62].

The retrieval augmented LLMs consist of two basic parts: generator and retriever. According to previous studies [32, 41, 83, 96], the retrieval augmentation effect is highly influenced by the retrieved content. In practice, there are two common types of retrievers. One is to leverage the general purpose retrievers, such as sparse models

like BM25 [69], and dense models, like DPR [37], contriever [30], E5 [81], BGE [89], OpenAI text embedding [56]. The other option is develop task-specific retriever, e.g., AAR for knowledge enhancement [96], LLM-R [85] for in-context learning. The general purpose methods are praised for their generality and simplicity for usage, but may suffer from an inferior retrieval quality. In contrast, the task-specific ones can better fit one scenario, but fall short in transferability. Compared with the existing works, LLM-Embedder unifies the generality and speciality: it comprehensive supports all major retrieval augmentation needs of LLMs, meanwhile achieving the leading performance in every application scenario.

• **Dense retrieval.** Dense retrieval leverages latent representation of texts, i.e. embeddings, to search for relevant information from a vector DB. In recent years, it has grown into a major paradigm of information retrieval. The success of dense retrieval can attribute to several reasons. The first and foremost driving force is the development of pre-trained language models[22, 45, 65], where the textual data can be represented in a highly expressive manner. The general pre-trained models are further improved by the retrieval-oriented ones [46, 81], which better establish the sentence-level representation capability during the pre-training stage. The second factor is the advancement of contrastive learning. On one hand, there has been a major upgrade of negative sampling, where massive [30, 37] and sufficiently hard samples [92] are utilized to help with the embedding’s discriminativeness. On the other hand, the training objective is improved as well. Instead of simply learning from hard labels, the embedding models are made to distill knowledge from a more precise ranking model [29, 63, 90]. This notably facilitates the embedding model to encode fine-grained semantic relationships. Thirdly, the generality becomes increasingly emphasized in these days, where embeddings need to handle a wide variety of application scenarios. For this purpose, people come up with many different strategies, e.g., data augmentation [42, 80], domain adaptation [36, 95], instruction-based fine-tuning [5, 76], which help the model to better handle diverse tasks. These factors are incorporated and optimized while developing our training recipe, which results in the empirical competitiveness of LLM-Embedder.

5 CONCLUSION

In this study, we introduce LLM-Embedder, a novel model designed to enhance the retrieval augmentation of LLMs in a variety of scenarios. Our model integrates four key retrieval capabilities: knowledge, memory, example, and tool, which boost LLMs' performance in dealing with knowledge-intensive tasks, long-context modeling, in-context learning, and tool learning. To optimize LLM-Embedder's performance in such diverse scenarios, we've refined our training workflow from multiple aspects, including reward from LLM, homogeneous negative sampling, instruction based fine-tuning, and stabilized distillation. Our experiments show LLM-Embedder's empirical advantages over both general and task-specific embedding models, which highlights its effectiveness as a foundational building-block to support the retrieval augmentation of LLMs.

REFERENCES

- [1] 2023. AquilaChat-7B. <https://huggingface.co/BAAI/AquilaChat-7B/>.
- [2] Chenxin An, Shansan Gong, Ming Zhong, Mukai Li, Jun Zhang, Lingpeng Kong, and Xipeng Qiu. 2023. L-Eval: Instituting Standardized Evaluation for Long Context Language Models.
- [3] Raviteja Anantha, Svitlana Vakulenko, Zhucheng Tu, Shayne Longpre, Stephen Pulman, and Srinivas Chappidi. 2020. Open-domain question answering goes conversational via question rewriting.
- [4] Akari Asai, Sewon Min, Zexuan Zhong, and Danqi Chen. 2023. Retrieval-based Language Models and Applications. , 41–46 pages.
- [5] Akari Asai, Timo Schick, Patrick Lewis, Xilun Chen, Gautier Izacard, Sebastian Riedel, Hannaneh Hajishirzi, and Wen-tau Yih. 2022. Task-aware retrieval with instructions.
- [6] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen Technical Report. *arXiv preprint arXiv:2309.16609* (2023).
- [7] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. 2022. Constitutional ai: Harmlessness from ai feedback.
- [8] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. 2023. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding.
- [9] Baichuan. 2023. Baichuan 2: Open Large-scale Language Models. *arXiv preprint arXiv:2309.10305* (2023). <https://arxiv.org/abs/2309.10305>
- [10] Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, Holy Lovenia, Ziwei Ji, Tiezheng Yu, Willy Chung, et al. 2023. A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity.
- [11] Luisa Bentivogli, Bernardo Magnini, Ido Dagan, Hoa Trang Dang, and Danilo Giampiccolo. 2009. The Fifth PASCAL Recognizing Textual Entailment Challenge. In *Proceedings of the Second Text Analysis Conference, TAC 2009, Gaithersburg, Maryland, USA, November 16-17, 2009*. NIST. https://tac.nist.gov/publications/2009/additionalpapers/RTE5_overview.proceedings.pdf
- [12] Amanda Bertsch, Uri Alon, Graham Neubig, and Matthew R Gormley. 2023. Unlimiformer: Long-range transformers with unlimited length input.
- [13] Sumithra Bhakthavatsalam, Daniel Khashabi, Tushar Khot, Bhavana Dalvi Mishra, Kyle Richardson, Ashish Sabharwal, Carissa Schoenick, Oyvind Tafjord, and Peter Clark. 2021. Think you have Solved Direct-Answer Question Answering? Try ARC-DA, the Direct-Answer AI2 Reasoning Challenge. *CoRR* abs/2102.03315 (2021). <https://arxiv.org/abs/2102.03315>
- [14] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about Physical Commonsense in Natural Language. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020*.
- [15] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. 2022. Improving language models by retrieving from trillions of tokens. , 2206–2240 pages.
- [16] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing, EMNLP 2015, Lisbon, Portugal, September 17-21, 2015*, Lluís Màrquez, Chris Callison-Burch, Jian Su, Daniele Pighin, and Yuval Marton (Eds.). The Association for Computational Linguistics, 632–642. <https://doi.org/10.18653/v1/d15-1075>
- [17] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. , 1877–1901 pages.
- [18] Daixuan Cheng, Shaohan Huang, Junyu Bi, Yuefeng Zhan, Jianfeng Liu, Yujing Wang, Hao Sun, Furu Wei, Denvy Deng, and Qi Zhang. 2023. UPRISE: Universal Prompt Retrieval for Improving Zero-Shot Evaluation.
- [19] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. Palm: Scaling language modeling with pathways.
- [20] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Tamar Solorio (Eds.). Association for Computational Linguistics, 2924–2936. <https://doi.org/10.18653/v1/n19-1300>
- [21] DataCanary, hilfalkaff, Lili Jiang, Meg Risdal, Nikhil Dandekar, and tomtung. 2017. Quora Question Pairs. <https://kaggle.com/competitions/quora-question-pairs>
- [22] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding.
- [23] William B. Dolan and Chris Brockett. 2005. Automatically Constructing a Corpus of Sentential Paraphrases. In *Proceedings of the Third International Workshop on Paraphrasing, IWP@IJCNLP 2005, Jeju Island, Korea, October 2005, 2005*. Asian Federation of Natural Language Processing. <https://aclanthology.org/105-5002/>
- [24] Ondrej Dusek, David M. Howcroft, and Verena Rieser. 2019. Semantic Noise Matters for Neural Natural Language Generation. In *Proceedings of the 12th International Conference on Natural Language Generation, INLG 2019, Tokyo, Japan, October 29 - November 1, 2019*, Kees van Deemter, Chenghua Lin, and Hiroya Takamura (Eds.). Association for Computational Linguistics, 421–426. <https://doi.org/10.18653/v1/W19-8652>
- [25] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. 2020. The pile: An 800gb dataset of diverse text for language modeling.
- [26] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The Pile: An 800GB Dataset of Diverse Text for Language Modeling.
- [27] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. Retrieval augmented language model pre-training. , 3929–3938 pages.
- [28] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding.
- [29] Sebastian Hofstätter, Sheng-Chieh Lin, Jheng-Hong Yang, Jimmy Lin, and Allan Hanbury. 2021. Efficiently teaching an effective dense retriever with balanced topic aware sampling. , 113–122 pages.
- [30] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Unsupervised dense information retrieval with contrastive learning.
- [31] Gautier Izacard and Edouard Grave. 2020. Leveraging passage retrieval with generative models for open domain question answering.
- [32] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2022. Few-shot learning with retrieval augmented language models.
- [33] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. , 38 pages.
- [34] Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation.
- [35] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [36] Constantinos Karouzos, Georgios Paraskevopoulos, and Alexandros Potamianos. 2021. UDALM: Unsupervised domain adaptation through language modeling.
- [37] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering.
- [38] Daniel Khashabi, Snigdha Chaturvedi, Michael Roth, Shyam Upadhyay, and Dan Roth. 2018. Looking Beyond the Surface: A Challenge Set for Reading Comprehension over Multiple Sentences. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2018, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 1 (Long Papers)*, Marilyn A. Walker, Heng Ji, and Amanda Stent (Eds.). Association for Computational Linguistics, 252–262. <https://doi.org/10.18653/v1/n18-1023>
- [39] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. , 453–466 pages.
- [40] Hector J. Levesque. 2011. The Winograd Schema Challenge. In *Logical Formalizations of Commonsense Reasoning, Papers from the 2011 AAAI Spring Symposium, Technical Report SS-11-06, Stanford, California, USA, March 21-23, 2011*. AAAI. <http://www.aaai.org/ocs/index.php/SSS/SSS11/paper/view/2502>
- [41] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. , 9459–9474 pages.
- [42] Patrick Lewis, Yuxiang Wu, Linqing Liu, Pasquale Minervini, Heinrich Küttler, Aleksandra Piktus, Pontus Stenetorp, and Sebastian Riedel. 2021. Paq: 65 million probably-asked questions and what you can do with them. , 1098–1115 pages.
- [43] Bill Yuchen Lin, Ming Shen, Wangchunshu Zhou, Pei Zhou, Chandra Bhagavatula, Yejin Choi, and Xiang Ren. 2020. CommonGen: A Constrained Text Generation Challenge for Generative Commonsense Reasoning. In *Conference on*

- Automated Knowledge Base Construction, AKBC 2020, Virtual, June 22-24, 2020*, Dipanjan Das, Hannaneh Hajishirzi, Andrew McCallum, and Sameer Singh (Eds.). <https://www.akbc.ws/2020/papers/yuD2q50HWv>
- [44] Xiao Liu, Hanyu Lai, Hao Yu, Yifan Xu, Aohan Zeng, Zhengxiao Du, Peng Zhang, Yuxiao Dong, and Jie Tang. 2023. WebGLM: Towards An Efficient Web-Enhanced Question Answering System with Human Preferences.
- [45] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach.
- [46] Zheng Liu and Yingxia Shao. 2022. Retromae: Pre-training retrieval-oriented transformers via masked auto-encoder.
- [47] Aman Madaan, Niket Tandon, Peter Clark, and Yiming Yang. 2022. Memory-assisted prompt editing to improve gpt-3 after deployment.
- [48] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi. 2022. When not to trust language models: Investigating effectiveness and limitations of parametric and non-parametric memories.
- [49] Kelong Mao, Hongjin Qian, Fengran Mo, Zhicheng Dou, Bang Liu, Xiaohua Cheng, and Zhao Cao. 2023. Learning Denoised and Interpretable Session Representation for Conversational Search. , 3193–3202 pages.
- [50] Kelong Mao, Hongjin Qian, Fengran Mo, Zhicheng Dou, Bang Liu, Xiaohua Cheng, and Zhao Cao. 2023. Learning Denoised and Interpretable Session Representation for Conversational Search. In *Proceedings of the ACM Web Conference 2023* (Austin, TX, USA) (WWW '23). Association for Computing Machinery, New York, NY, USA, 3193–3202. <https://doi.org/10.1145/3543507.3583265>
- [51] Grégoire Mialon, Roberto Dessi, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, et al. 2023. Augmented language models: a survey.
- [52] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, 2381–2391. <https://doi.org/10.18653/v1/d18-1260>
- [53] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2022. MTEB: Massive text embedding benchmark.
- [54] Linyong Nan, Dragomir R. Radev, Rui Zhang, Amrit Rau, Abhinand Sivaprasad, Chiachun Hsieh, Xiangru Tang, Aadit Vyas, Neha Verma, Pranav Krishna, Yangxiaokang Liu, Nadia Irwanto, Jessica Pan, Faiaz Rahman, Ahmad Zaidi, Mutethia Mutuma, Yasin Tarabar, Ankit Gupta, Tao Yu, Yi Chern Tan, Xi Victoria Lin, Caiming Xiong, Richard Socher, and Nazneen Fatema Rajani. 2021. DART: Open-Domain Structured Data Record to Text Generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*, Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tür, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tannoy Chakraborty, and Yichao Zhou (Eds.). Association for Computational Linguistics, 432–447. <https://doi.org/10.18653/v1/2021-naacl-main.37>
- [55] Courtney Napoles, Matthew R. Gormley, and Benjamin Van Durme. 2012. Annotated Gigaword. In *Proceedings of the Joint Workshop on Automatic Knowledge Base Construction and Web-scale Knowledge Extraction, AKBC-WEKEX@NAACL-HLT 2012, Montréal, Canada, June 7-8, 2012*, James Fan, Raphael Hoffman, Aditya Kalyanpur, Sebastian Riedel, Fabian M. Suchanek, and Partha Pratim Talukdar (Eds.). Association for Computational Linguistics, 95–100. <https://aclanthology.org/W12-3018/>
- [56] Arvind Neelakantan, Tao Xu, Raul Puri, Alec Radford, Jesse Michael Han, Jerry Tworek, Qiming Yuan, Nikolas Tezak, Jong Wook Kim, Chris Hallacy, et al. 2022. Text and code embeddings by contrastive pre-training.
- [57] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. Ms marco: A human-generated machine reading comprehension dataset.
- [58] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. , 27730–27744 pages.
- [59] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, et al. 2020. KILT: a benchmark for knowledge intensive language tasks.
- [60] Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Dmitry Okhonko, Samuel Broscheit, Gautier Izacard, Patrick Lewis, Barlas Oğuz, Edouard Grave, Wen-tau Yih, et al. 2021. The web is your oyster-knowledge-intensive NLP against a very large web corpus.
- [61] Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Yufei Huang, Chaojun Xiao, Chi Han, et al. 2023. Tool learning with foundation models.
- [62] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis.
- [63] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Xin Zhao, Daxiang Dong, Hua Wu, and Haifeng Wang. 2020. RocketQA: An optimized training approach to dense passage retrieval for open-domain question answering.
- [64] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, Chloe Hillier, and Timothy P Lillicrap. 2019. Compressive Transformers for Long-Range Sequence Modelling. <https://arxiv.org/abs/1911.05507>
- [65] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. , 5485–5551 pages.
- [66] Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know What You Don't Know: Unanswerable Questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, ACL 2018, Melbourne, Australia, July 15-20, 2018, Volume 2: Short Papers*, Iryna Gurevych and Yusuke Miyao (Eds.). Association for Computational Linguistics, 784–789. <https://doi.org/10.18653/v1/P18-2124>
- [67] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, EMNLP 2016, Austin, Texas, USA, November 1-4, 2016*, Jian Su, Xavier Carreras, and Kevin Duh (Eds.). The Association for Computational Linguistics, 2383–2392. <https://doi.org/10.18653/v1/d16-1264>
- [68] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks.
- [69] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. , 333–389 pages.
- [70] Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S. Gordon. 2011. Choice of Plausible Alternatives: An Evaluation of Commonsense Causal Reasoning. In *Logical Formalizations of Commonsense Reasoning, Papers from the 2011 AAAI Spring Symposium, Technical Report SS-11-06, Stanford, California, USA, March 21-23, 2011*. AAAI. <http://www.aaai.org/ocs/index.php/SSS/SSS11/paper/view/2418>
- [71] Ohad Rubin and Jonathan Berant. 2023. Long-range Language Modeling with Self-retrieval.
- [72] Tapan Sahni, Chinmay Chandak, Naveen Reddy Chedeti, and Manish Singh. 2017. Efficient Twitter sentiment classification using subjective distant supervision. In *9th International Conference on Communication Systems and Networks, COMSNETS 2017, Bengaluru, India, January 4-8, 2017*. IEEE, 548–553. <https://doi.org/10.1109/COMSNETS.2017.7945451>
- [73] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. WinoGrande: an adversarial winograd schema challenge at scale. *Commun. ACM* 64, 9 (2021), 99–106. <https://doi.org/10.1145/34734381>
- [74] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools.
- [75] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng, and Christopher Potts. 2013. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, EMNLP 2013, 18-21 October 2013, Grand Hyatt Seattle, Seattle, Washington, USA, A meeting of SIGDAT, a Special Interest Group of the ACL*. ACL, 1631–1642. <https://aclanthology.org/D13-1170/>
- [76] Hongjin Su, Jungo Kasai, Yizhong Wang, Yushi Hu, Mari Ostendorf, Wen-tau Yih, Noah A Smith, Luke Zettlemoyer, Tao Yu, et al. 2022. One embedder, any task: Instruction-finetuned text embeddings.
- [77] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models.
- [78] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models.
- [79] Lewis Tunstall, Leandro Von Werra, and Thomas Wolf. 2022. Natural language processing with transformers.
- [80] Kexin Wang, Nandan Thakur, Nils Reimers, and Iryna Gurevych. 2021. Gpl: Generative pseudo labeling for unsupervised domain adaptation of dense retrieval.
- [81] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Simlm: Pre-training with representation bottleneck for dense passage retrieval.
- [82] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training.
- [83] Liang Wang, Nan Yang, and Furu Wei. 2023. Learning to Retrieve In-Context Examples for Large Language Models.
- [84] Tianshi Wang, Li Liu, Huaxiang Zhang, Long Zhang, and Xiuxiu Chen. 2020. Joint Character-Level Convolutional and Generative Adversarial Networks for Text Classification. *Complex* 2020 (2020), 8516216:1–8516216:11. <https://doi.org/10.1155/2020/8516216>
- [85] Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. 2023. Augmenting Language Models with Long-Term Memory.

- [86] Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. 2021. Finetuned language models are zero-shot learners.
- [87] Adina Williams, Nikita Nangia, and Samuel R. Bowman. 2018. A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2018, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 1 (Long Papers)*, Marilyn A. Walker, Heng Ji, and Amanda Stent (Eds.). Association for Computational Linguistics, 1112–1122. <https://doi.org/10.18653/v1/n18-1101>
- [88] Yuhuai Wu, Markus N Rabe, DeLesley Hutchins, and Christian Szegedy. 2022. Memorizing transformers.
- [89] Shitao Xiao and Zheng Liu. 2023. BAAI General Embedding. <https://github.com/FlagOpen/FlagEmbedding>
- [90] Shitao Xiao, Zheng Liu, Weihao Han, Jianjin Zhang, Defu Lian, Yeyun Gong, Qi Chen, Fan Yang, Hao Sun, Yingxia Shao, et al. 2022. Distill-vq: Learning retrieval oriented vector quantization by distilling knowledge from dense embeddings. , 1513–1523 pages.
- [91] Shitao Xiao, Zheng Liu, Yingxia Shao, Defu Lian, and Xing Xie. 2021. Matching-oriented product quantization for ad-hoc retrieval.
- [92] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. 2020. Approximate nearest neighbor negative contrastive learning for dense text retrieval.
- [93] Jing Xu, Arthur Szlam, and Jason Weston. 2021. Beyond goldfish memory: Long-term open-domain conversation.
- [94] Jing Xu, Arthur Szlam, and Jason Weston. 2022. Beyond Goldfish Memory: Long-Term Open-Domain Conversation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, 5180–5197. <https://doi.org/10.18653/v1/2022.acl-long.356>
- [95] Yue Yu, Chenyan Xiong, Si Sun, Chao Zhang, and Arnold Overwijk. 2022. Coco-dr: Combating distribution shifts in zero-shot dense retrieval with contrastive and distributionally robust learning.
- [96] Zichun Yu, Chenyan Xiong, Shi Yu, and Zhiyuan Liu. 2023. Augmentation-Adapted Retriever Improves Generalization of Language Models as Generic Plug-In.
- [97] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence?. In *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, Anna Korhonen, David R. Traum, and Lluís Màrquez (Eds.). Association for Computational Linguistics, 4791–4800. <https://doi.org/10.18653/v1/p19-1472>
- [98] Rui Zhang and Joel R. Tetreault. 2019. This Email Could Save Your Life: Introducing the Task of Email Subject Line Generation. In *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, Anna Korhonen, David R. Traum, and Lluís Màrquez (Eds.). Association for Computational Linguistics, 446–456. <https://doi.org/10.18653/v1/p19-1043>
- [99] Xiang Zhang, Junbo Jake Zhao, and Yann LeCun. 2015. Character-level Convolutional Networks for Text Classification. In *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*, Corinna Cortes, Neil D. Lawrence, Daniel D. Lee, Masashi Sugiyama, and Roman Garnett (Eds.). 649–657. <https://proceedings.neurips.cc/paper/2015/hash/250cf8b51c773f3f8dc8b4be867a9a02-Abstract.html>
- [100] Yuan Zhang, Jason Baldridge, and Luheng He. 2019. PAWS: Paraphrase Adversaries from Word Scrambling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, 1298–1308. <https://doi.org/10.18653/v1/n19-1131>
- [101] Bartosz Piotrowski Zhangir Azerbayev, Edward Ayers. 2008. Proof-Pile. <https://huggingface.co/datasets/hoskinson-center/proof-pile/>. [Online; accessed 19-July-2008].

Table 5: Statistics of Multi-Session Chat: the average number of turns of dialogue and the average token number per utterance in training/testing.

Split	#Sample	# History Turn	Utterance Length
Train	48925	14	33
Test	2763	27	44

A DATASET DETAILS

A.1 Knowledge Enhancement

A.1.1 MMLU. MMLU is a multitask language understanding dataset with 14042 multi-choice questions, spanning 57 diverse subtasks, such as linear algebra, computer science, etc. For retrieval augmentation, we retrieve 3 passages from MSMARCO Passage [57] collection, which consists of 8841823 passages from the web. We use the official prompt template for MMLU evaluation, which is shown in A.1, and prepend the retrieved passages to the question. Since our experiments are based on the Chat fine-tuned model, we omit the few-shot examples on MMLU. We select the option with the highest likelihood (from A, B, C, D) as the answer from LLM.

Prompt A.1: MMLU (Zero-Shot)

Knowledge:
 <Passage 1>
 <Passage 2>
 <Passage 3>

The following are multiple choice questions (with answers) about <subject>.

<Question>
 A. <Option 1>
 B. <Option 2>
 C. <Option 3>
 D. <Option 4>
 Answer:

A.1.2 PopQA. PopQA is a Wikipedia-entity-centric question-answering dataset that covers 14267 questions about popular and long-tail entities. For retrieval augmentation, we retrieve 3 passages from Wikipedia 2019 dump preprocessed by [37], which contains 21051324 passages with 100 tokens each. We use the official prompt template and few-shot evaluation strategy on PopQA, shown in A.2. There are 15 few-shot demonstrations per question, each coming from one distinct relationship. The model conducts greedy generation until \n token. The produced answer is regarded as correct if it contains any of the pre-defined answers in the dataset.

Prompt A.2: PopQA

Knowledge:
 <Passage 1>
 <Passage 2>
 <Passage 3>

Q: <Question 1> A: <Answer 1>
 Q: <Question 2> A: <Answer 2>
 ...
 Q: <Question 15> A: <Answer 15>
 Q: <Question> A:

A.1.3 Training. Both MMLU and PopQA are only used for evaluation. To train the retriever towards acquiring useful knowledge for the LLM, we use two popular retrieval datasets: MSMARCO [57] and NQ [39]. Their statistics are shown in Table 9.

A.2 Long-Context Modeling

A.2.1 Long Conversation. We leverage the Multi-Session Chat dataset [94] to evaluate the LLM’s performance on the long conversation. MSC is a dialogue dataset containing multiple sessions between two speakers. The statistics of the Multi-Session Chat dataset are reported in Table 5. Each dialogue turn is this format: “Speaker 1: xxx\nSpeaker 2: xxx” and consecutive turns are split by \n. The token-level perplexity is evaluated by the response from Speaker 2. For *None* baseline, we only input LLM the last dialogue turn. For *Recent* baselines, the most recent 2 dialogue turns are input to LLM. For retrieval augmentation, one dialogue turn is retrieved from the entire history and is concatenated in front of the last turn.

Table 6: Statistics of long-range language modeling datasets.

Dataset	#Train Sample	#Test Sample	Length
Books3	10000	1000	101010
Arxiv	10000	757	26735
CodeParrot	10000	1000	217364
PG19	n.a.	1000	90447

A.2.2 Long-Range Language Modeling. We utilize four popular long-range language modeling datasets to evaluate LLM’s performance on long sequences: *Books3* includes various literary works from different domains, and *PG19* contains books from Project Gutenberg. Both datasets are extracted from the Pile [26]. *Arxiv*, a.k.a. Proof-Pile [101], is a collection of mathematical preprints on arxiv. *CodeParrot* is a vast corpus of cleaned project code from Github. We concatenate the code of the same repository to obtain long enough text, resulting in 437079 samples in total. For all four datasets, we filter out text that’s shorter than 160k characters, then randomly sample 10000 for training and 1000 for testing. The PG19 dataset is held out and only used in testing. We summarize the statistics of four long-range language modeling datasets in Table 6.

In practice, we truncate all testing samples to 32768 tokens. The token-level perplexity is evaluated with batch size 1 on the last 1024 tokens (dubbed as target tokens). For *None* and *Recent* baselines, the last 2048 and 4096 tokens are fed into the model, respectively. For retrieval augmentation, the text is split into chunks with chunk size 128, and the last 2048 tokens are always fixed during evaluation. For each chunk in the target tokens, we retrieve 8 chunks and their continuation chunk (chunk size 128) from the previous 30720 tokens. The retrieved chunks are directly concatenated in front of the fixed 2048 tokens without delimiters.

A.3 In-Context Learning

The detailed information about in-context learning datasets is reported in Table 7. Particularly, there are two evaluation strategies:

Table 7: Detailed information of in-context learning datasets.

Dataset name	Category	#Train Sample	#Test Sample	Metric	Evaluation Strategy
ARC Challenge [13]	Close QA	1,117	1,165	Accuracy	Likelihood
ARC Easy [13]	Close QA	2,241	2,365	Accuracy	Likelihood
NQ [39]	Close QA	87,925	3,610	Exact Match	Generation
COPA [70]	Commonsense	400	100	Accuracy	Likelihood
HellaSwag [97]	Commonsense	39,905	10,042	Accuracy	Likelihood
PIQA [14]	Commonsense	16,113 (held out)	1,838	Accuracy	Likelihood
Winogrande [73]	Coreference	40,398	1,267	Accuracy	Likelihood
WSC [40]	Coreference	554	104	Accuracy	Likelihood
WSC273 [40]	Coreference	0 (held out)	273	Accuracy	Likelihood
CommonGen [43]	Data-to-text	67,389	4,018	ROUGE-L	Generation
DART [54]	Data-to-text	62,659	2,768	ROUGE-L	Generation
E2E NLG [24]	Data-to-text	33,525	1,847	ROUGE-L	Generation
MNLI (m) [87]	NLI	392,702	9,815	Accuracy	Likelihood
MNLI (mm) [87]	NLI	392,702	9,832	Accuracy	Likelihood
RTE [11]	NLI	2,490	277	Accuracy	Likelihood
SNLI [16]	NLI	549,367	9,824	Accuracy	Likelihood
QNLI [66]	NLI	104,743 (held out)	5,463	Accuracy	Likelihood
MRPC [23]	Paraphrase	3,668	408	Accuracy	Likelihood
PAWS [100]	Paraphrase	49,401	8,000	Accuracy	Likelihood
QQP [21]	Paraphrase	363,846	40,430	Accuracy	Likelihood
BoolQ [20]	Reading Comp.	9,427	3,270	Accuracy	Likelihood
MultiRC [38]	Reading Comp.	27,243	4,848	F1	Likelihood
OpenBook QA [52]	Reading Comp.	4,957	500	Accuracy	Likelihood
SQuAD v1 [67]	Reading Comp.	87,599	10,570	Exact Match	Generation
Sentiment140 [72]	Sentiment	1,600,000	359	Accuracy	Likelihood
SST2 [75]	Sentiment	67,349	872	Accuracy	Likelihood
Yelp [84]	Sentiment	490,456 (held out)	33,285	Accuracy	Likelihood
AESLC [98]	Summarize	13,181	1,750	ROUGE-L	Generation
AGNews [99]	Summarize	120,000	7,600	Accuracy	Likelihood
Gigaword [55]	Summarize	2,044,465	730	ROUGE-L	Generation
Total	n.a.	6.3M	177k	n.a.	n.a.
Total (sampled)	n.a.	591k	177k	n.a.	n.a.

Table 8: Statistics of tool learning and conversational search datasets.

Dataset	#Train Sample	#Test Sample	#Corpus
ToolBench	87322	100	10439
QReCC	29596	8209	54573064

Likelihood and *Generation*. The former means we score each candidate option with the likelihood of LLM when there are available options (e.g. Yes and No on the WSC dataset), and pick the one with the highest score; The latter means we let LLM perform greedy generation without sampling. Following [83], we randomly sample at most 30000 instances from each dataset for training, and hold out 4 datasets from training. However, different from their evaluation, we keep the top-8 retrieved examples as is no matter if they belong to the same task as the input instruction or not.

A.4 Tool Learning and Conversational Search

A.4.1 Tool Learning. We use the ToolBench [62] dataset to evaluate the performance of tool retrieval, where the retriever takes in a user request, and searches for a helpful tool according to its description. The statistics of ToolBench are reported in Table 8.

A.4.2 Conversational Search. We employ the popular QReCC dataset [3] to evaluate the performance of conversational search. Specifically, there is a short conversation followed by a “contextualized” query in each sample. The retriever takes in the *concatenation of the whole conversation and the query* to find the relevant passage in the given corpus. The statistics of QReCC are reported in Table 8.

A.5 Summary of Multi-Task Training Data

We summarize the dataset details for training in Table 9. Notably, we repeat the ToolBench data in every epoch because we find the retriever requires more epoch to converge on this single task.

Table 9: Dataset details for training.

Task	Dataset	#Train Sample	Repetition	Stablized Distillation	Reward Temperature
Question Answering	MSMARCO	400870	1	✓	1
	NQ	58622	1		1
In-Context Learning	–	591359	1	✓	1
Long Conversation	MSC	48925	1	✓	0.1
Long-Range Language Modeling	Books3	10000	1	✓	0.1
	Arxiv	10000	1		0.1
	CodeParrot	10000	1		0.1
Tool Learning	ToolBench	87322	2	✗	n.a.
Conversational Search	QReCC	29596	1	✗	n.a.
Total	n.a.	1333911	n.a.	n.a.	n.a.

Table 10: Instructions for each task.

Task	Input	Instruction
Question Answering	Query Key	Represent this query for retrieving relevant documents: Represent this document for retrieval:
In-Context Learning	Query Key	Convert this example into vector to look for useful examples: Convert this example into vector for retrieval:
Long Conversation	Query Key	Embed this dialogue to find useful historical dialogues: Embed this historical dialogue for retrieval:
Long-Range Language Modeling	Query Key	Embed this text chunk for finding useful historical chunks: Embed this historical text chunk for retrieval:
Tool Learning	Query Key	Transform this user request for fetching helpful tool descriptions: Transform this tool description for retrieval:
Conversational Search	Query Key	Encode this query and context for searching relevant passages: Encode this passage for retrieval:

Table 11: Hyper parameter settings for training.

#GPU	8×A100 (40G)
#Hard Negative	7
Batch Size Per GPU	100
Optimizer	AdamW
Learning Rate	5e-6
Weight Decay	0.01
Scheduler	Linear with Warm Up of 0.2
Max Steps	10000
Gradient Checkpointing	✓

B IMPLEMENTATION DETAILS

B.1 Instructions

We use diversified instructions to discriminate different tasks for the retriever. The instructions used for each task are shown in Table 10.

B.2 Training Settings

The hyper parameter settings for training LLM-Embedder are reported in Table 11. For evaluation, we use the Flat index from Faiss [35] when retrieving from an external corpus is required. We will release our code upon the acceptance of the paper.

C IMPACT OF LLM-EMBEDDER ON DIFFERENT LLMS

We evaluate the impact of LLM-Embedder different LLMS to validate its generalization ability. Specifically, we utilize Aquila-7B-Chat [1], Qwen-7B-Chat [6], Baichuan2-7B-Chat [9], and Llama-2-13B-Chat [78]. The results are shown in Table 12. Specifically, we compare two baselines: None, where LLM is used individually without retrieval augmentation; BGE, where LLM is augmented with retrieved knowledge, examples, and memory (introduced in Appendix A). We report the average accuracy for MMLU, accuracy for PopQA, average score for in-context learning, and perplexity for both Multi-Session Chat and Arxiv. Note that we do not replicate

Table 12: The impact of LLM-Embedder on different LLMs.

LLM	Retriever	MMLU	PopQA	ICL	MSC	Arxiv
Llama-2-7B-Chat	None	0.4599	0.2061	0.4645	19.3501	3.7647
	BGE	0.4896	0.4491	0.5974	14.2943	3.2912
	LLM-Embedder	0.4903	0.5052	0.6268	13.4832	3.2322
Aquila-7B-Chat	None	0.4499	0.2028	0.5145	16.0108	3.1204
	BGE	0.4832	0.3982	0.5732	14.1843	2.7914
	LLM-Embedder	0.4847	0.4405	0.5903	14.1836	2.7351
Qwen-7B-Chat	None	0.5561	0.2393	0.5346	21.0466	2.7888
	BGE	0.5787	0.4447	0.6329	16.2064	2.5165
	LLM-Embedder	0.5762	0.4782	0.6457	15.4524	2.4824
Baichuan2-7B-Chat	None	0.5226	0.2356	0.4907	18.9711	2.7510
	BGE	0.5534	0.4407	0.5960	16.0759	2.4440
	LLM-Embedder	0.5511	0.4848	0.6179	15.5890	2.4131
Llama-2-13B-Chat	None	0.5386	0.2886	0.4607	14.7334	3.2357
	BGE	0.5603	0.4595	0.6196	11.6875	2.9036
	LLM-Embedder	0.5580	0.5026	0.6439	11.5384	2.8540

the evaluation of tool learning and conversational search because their performances are directly measured by retrieval metrics.

We can observe that our conclusions in Section 3.2.2 still holds. First of all, retrieval from external world benefits LLM’s performance in all four scenarios, since the performance of the plain LLM (i.e. None) underperforms retrieval-augmented one (BGE and LLM-Embedder). Besides, our proposed LLM-Embedder is able to

generalize well and maintain its superiority over BGE on most datasets (PopQA and ICL in particular). An exception is MMLU, where LLM-Embedder is slightly outperformed by BGE when using Qwen, Baichuan, and Llama-2-13B. It seems that different LLMs utilize the same knowledge in different ways, thereby obtaining a little different results.